Department of Computer Science and Engineering
University of Puerto Rico
Mayagüez Campus
INSO 6006 – Database Systems Engineering
Fall 2026

Project #3: The B+tree Index

Due Date: November 16, 11:59 PM

Objectives

- Understand the B+tree: sorted leaves holding the data entries, separator pages routing every descent, and the order-d shape that keeps every leaf at the same depth.
- Implement the lecture's four operations — search, insert, erase, and verification — as recursive methods whose auxiliary functions carry split and underflow state upward.
- Understand index rebalancing: redistribution with a sibling versus merge, exact separators, and the root's exemption.
- Gain experience with C++ recursion over pinned pages, fixed-width encoded keys, and byte-level page views behind a buffer manager.

Overview

You will complete the index layer of a small database engine. The supplied project contains the complete engine — the TUP2 heap file, the buffer pool and manager, the block-file layer, the catalog with its atomic writer, the SQL layer and executors, the ISAM index, the B+tree page views, the bulk loader, the session, and all tests. The engine works end to end except for the B+tree's algorithm methods: the member methods declared in index/BPlusTree.h — documented there in full, with their lecture-slide references — are blank stubs in index/BPlusTree.cpp. Your task is to implement those methods until the supplied check suite passes. Do not modify the provided files.

The B+tree indexes one column of one table. Leaf pages hold sorted (key, rid) data entries chained doubly; separator pages hold the |P0| K1 | P1 | … | Km | Pm | index-entry form; block 0 is a header page carrying the magic, the key identification, the root, the first leaf, the level count, the entry count, and the head of a free-block list. Every non-root node holds between d and 2d keys, where d = floor(capacity / 2); the root is exempt. Every separator always equals the first key of the child it routes to — the exactness invariant your implementation must maintain.

Verification — verify_rec

Implement the integrity walk first: it is the net every later step gets caught by. Starting at the root, mark every visited page (a second visit is a "reachable twice" finding; a block beyond the file is a finding). A separator page checks its occupancy against the order rule (the root is exempt), its separators in ascending order, and recurses into its first child and every pair's child — each separator must equal its child's subtree minimum, which the recursion returns upward. A leaf checks its occupancy and entry order, judges every rid through the supplied occupied callback (dangling entries are findings), and feeds

the walked total and the leaf-depth range. The wrapper assembles the walk: the free list, block coverage, the leaf chain's global order and total, and the header agreement.

Search — tree_search, run_start, gather_run, locate_in_run

The descent follows the lecture's routing rule: pin one separator, take the child after the LAST separator less than or equal to the key, and recurse; the level count reaching one means the page is the landing leaf. Point lookup and ranges gather the equal-key run left to right through the doubly linked leaves: walk left from the landing leaf while the previous leaf's last key EQUALS the target (the exact boundary test — a greater-or-equal test over-walks), then collect forward, continuing into next_leaf while a leaf's last key equals the target. locate_in_run scans the same run for one exact (key, rid) pair and reports the holding leaf and position.

Insert — insert_leaf, insert_aux

The arriving entry descends by the routing rule to its leaf. A leaf with room within 2d takes a plain sorted insert. A full leaf (2d entries) splits per the lecture: this page keeps its first d entries, a new leaf takes the remaining d+1, the new leaf's smallest key is copied up as the separator, and the chain links both ways. The arriving entry then takes its place in the correct half — equal keys go right, where the routing sends their duplicates. A full separator page (2d pairs) splits by moving its middle key up, with its child becoming the new page's first child. When the old root splits, the wrapper allocates a new root above both halves and the height grows by exactly one.

Erase — erase_leaf, erase_rec, fix_child_underflow, borrow_from_left, borrow_from_right, merge_child

The exact (key, rid) entry is located in the equal-key run and removed with a within-page tail-shift — the vacated slot closes up. A leaf left with fewer than d entries underflows, and the repair follows the lecture: redistribute with a sibling that can lend (left preferred, then right), rotating the entry or the child-and-key pair through the parent's separator and rewriting that separator so it stays exact; otherwise merge the page with a sibling and drop the separator from the parent. An underflow at the parent's level propagates the same way, and a root left with a single child is dropped, shrinking the height by exactly one. An emptied child's boundaries move too — its parent separator follows the subtree's new minimum.

Tasks for Project 3

You will receive the Git repository with the following method bodies blank in index/BPlusTree.cpp. Implement these methods:

```
verify_rec
tree_search
run_start
gather_run
locate_in_run
insert_leaf
insert_aux
erase_leaf
erase_rec
fix_child_underflow
```

`borrow_from_left`
`borrow_from_right`
`merge_child`

All header files, the page views (index/BptPages.cpp), the bulk build, the public wrappers, the ISAM index, the heap file, the buffer manager, the catalog, the SQL layer, the check programs, and the build configuration are provided. Do not modify the provided files.

Testing and Submission

Build the project with CMake and run the supplied check executable or CTest suite. Test built trees of several sizes (heights one through four), duplicates that span leaf boundaries, ranges with adjacent duplicates, height growth at the first separator overflow, height collapse as merges cascade, verification cleanliness after every operation, and the randomized insert/erase sequences against the shadow model. The storage, SQL, ISAM, and loader suites must stay green throughout — they are the provided infrastructure's regression net. Submit only index/BPlusTree.cpp unless your instructor specifies a different submission procedure.

PROJECT DUE DATE: November 16, 2026 11:59 PM